Informatics

The R language and system
with programming examples
Part 2 - Loops

Claudio Sartori
Department of Computer Science and Engineering
claudio.sartori@unibo.it
https://www.unibo.it/sitoweb/claudio.sartori/

Repetition

- One of the base constructs necessary when writing algorithms is the *repetition (loop)*
- All the programming languages, including R, have several constructs for the repetition

`for`

`while`

`repeat`

while

- while is a reserved word of the language
- the logical expression can evaluate to TRUE or FALSE
- if the logical expression is true the body is executed and the logical expression is evaluated again
- if the logical expression is false in the beginning, the body is not executed

```
...instructions including assignment
...of values to the variables
...included in the logical expression
while (logical expression){
  ...sequence of statements
  ...including change of values
  ...of the variables included
  ...in the logical expression
}
```

for

Necessary to specify

- When to stop the repetition
- What is varying along with the repetition
- What instructions to repeat

```
for (var in seq){
    sequence of statements
    …
}
```

for (ii)

- *var* is a variable name
- *seq* is a sequence of values of any type
- *var* gets all the values in *seq* and for each of them the sequence of statements is executed
- After the last value the statement following the for is executed

```
for (var in seq){
    sequence of statements
    …
}
...
```

Sequences (i)

- A closed sequence of contiguous integers
- A sequence of numbers with a start value, an end value and a step
  - start, end, step are expressions
  - the end and can be reached or not, depending on the start and step values

```
> 1:8
[1] 1 2 3 4 5 6 7 8


> seq(1,2,0.3)
 [1] 1.0 1.3 1.6 1.9
> seq(0,2*pi,pi/2)
[1] 0.000000 1.570796 3.141593
4.712389 6.283185

for (x in seq(0,2*pi,pi/2)){
  cat(cos(x),"\n")
}
1
6.123234e-17
-1
-1.83697e-16
1
```

Should be 0, the computed value is not exact because of limited *precision*

Sequences (ii)

- The first n integers
  - if n is 0 the sequence is empty

- All the positions of a vector

```
> seq_len(4)
[1] 1 2 3 4
> seq_len(0)
Integer(0) # empty sequence
```

```
> a <- c(30,50,80)
> seq_along(a)
[1] 1 2 3
```

The compound interest

algo to R prog

```
# Compound interest for n years

# Use variables A, n, r, Af
# Read A, n, r
# Af <- A
# Repeat n times
#    Af <- Af * (1 + r)
# Write Af
#
```

```
A <- as.double(readline(
     "Insert the initial amount  "))
r <- as.double(readline(
     "Insert the interest rate   "))
n <- as.integer(readline(
     "Insert the number of years "))
Af <- A
for (i in 1:n){
  Af <- Af*(1+r)
}
cat("Starting value  ",A, "\n")
cat("Interest rate   ",r, "\n")
cat("Number of years ",n, "\n")
cat("Final amount    ",Af,"\n")
```

Exercises: writing algorithms

algorithms

a) Compute the product of two values **m** and **n** using only the *sum* operation
b) Convert a string of bits into a decimal integer
c) Compute the arithmetic mean of **n** numbers read from input
d) Compute the factorial of **n**
e) Compute the value of e^x with the MacLaurin series

Compute the product of two values **m** and **n** using only the *sum* operation

- Use m, n, p
- Input m, n
- p <- 0
- Repeat n times
  - p <- p + m
- Output m, n, p

is this solution valid for any pair of numbers **m**, **n**?

```
m <- as.double(readline(
    'insert a number'))
n <- as.integer(readline(
    'insert an integer '))
p <- 0
for (i in 1:n){
   p <- p + m
}
cat(m,' times ', n, ' gives ',p)
```

Compute the product of two values **m** and **n** using only the *sum* operation

- Use m, n, p
- Input m, n
- p <- 0
- Repeat n times
  - p <- p + m
- Output m, n, p

The solution is valid for any **m** numeric and for **n** positive integer

```
m <- as.double(readline(
    'insert a number'))
n <- as.integer(readline(
    'insert an integer '))
p <- 0
for (i in 1:n){
   p <- p + m
}
cat(m,' times ', n, ' gives ',p)
```

Limits of a solution

- when you write an algorithm (and then translate it into a program) you must be aware of the limits of the solution:
  - what are the *valid* input values?
    - i.e. the input values for which the program produces correct results

Convert a string of bits into a decimal integer

Bits to integer - variables

```
# Uses
# B - string of 0 and 1, representing a binary number
# l - number of characters in B
# pv - position value: value of each position in B,
  it is a power # of 2, starting from 0 right to left
# p - current position examined in B
# b - bit of the current position
# d - decimal value, equivalent to B, given in output
```

Bits to integer – Algorithm

```
- read from input the binary string and store it in B
- store in l the number of characters in B (nchar(B))
- store in pv the value of the rightmost position, which is 1
- store in p the value of l, it will be the starting position
  to be considered, then will go right to left
- store 0 in d, the output will be computed with additions
- repeat while p is more than 0
  - store in b the p-th character of B, converted into integer
  - store in d the computation d + b * pv
  - store in pv the computation pv * 2
  - decrease p by 1
- print d
```

Compute the value of e^x with the MacLaurin series

Questions

algorithms

From mathematics we know that

$$e^x = 1 + x + x^2/2! + x^3/3! + \ldots$$

1. How can we compute the results of this sum?
   1. How can we compute all the terms to be added
2. How many terms should we add?
3. How many variables do we need?
4. Do we need to store all the terms?
5. Is there an easy way to compute one term, given the previous one?

algorithms

Computing the terms

$$e^x = \underset{t_0}{1} + \underset{t_1}{x} + \underset{t_2}{x^2/2!} + \underset{t_3}{x^3/3!} + \ldots$$

...

- Let's try to compute t_2/t_1

  x/2

- Let's try to compute t_3/t_2

  x/3

- In general, $t_i \leftarrow t_{i-1} * x / i$
- But we do not need the old value anymore, therefore

  $t \leftarrow t * x / i$

algorithms

Adding the terms

$$e^x = \underset{t_0}{1} + \underset{t_1}{x} + \underset{t_2}{x^2/2!} + \underset{t_3}{x^3/3!} + \ldots$$

...

- Each term will be added to the variable containing the sum

```
ex <- ex + t
```

- This operation will be repeated, according to the number of terms that will be used

```
Repeat n times varying i from 1 to n
    t ←t * x / i
    ex ← ex + t
```

algorithms

Initialization

$$e^x = \underset{t_0}{1} + \underset{t_1}{x} + \underset{t_2}{x^2/2!} + \underset{t_3}{x^3/3!} + \underset{\ldots}{\ldots}$$

- Many iterative computations contain a step like this

t ← f(t)

- What should be the starting value?
- Rule of thumb
  - What should be the correct value in case of 0 repetitions?

```
t ← 1
ex ← t
```

Putting things together

algorithms

$$e^x = \underset{t_0}{1} + \underset{t_1}{x} + \underset{t_2}{x^2/2!} + \underset{t_3}{x^3/3!} + \ldots$$

...

```
Use x, n, t, ex,
Input x and n
t ← 1
ex ← t
Repeat n times varying i from 1 to n
   t ← t * x / i
   ex ← ex + t
Output ex
```

How many terms do we really need?

algorithms

$$e^x = \underset{t_0}{1} + \underset{t_1}{x} + \underset{t_2}{x^2/2!} + \underset{t_3}{x^3/3!} + \ldots$$

...

- For x ≠ 0 the absolute value of terms is monotonically decreasing
- We could stop the computation when the term contribution is less than a given *precision*

t/ex < p

- We need a repetition *based on a condition* instead of one based on a sequence of values

Repeat *if condition is true*

Repetition based on condition

algorithms

$$e^x = \underset{t_0}{1} + \underset{t_1}{x} + \underset{t_2}{x^2/2!} + \underset{t_3}{x^3/3!} + \ldots$$

...

Use x, t, ex, p
Input x and p
t ← 1
ex ← t
Repeat if the absolute value of t/ex >= p
    t ← t * x / i
    ex ← ex + t
Output ex

What's missing?

https://app.wooclap.com/URMCJL

Repetition based on condition

Necessary to specify

- When to continue the repetition
  - Expressed as a logical expression
- What instructions to repeat
  - Must include some change that can make false the logical expression used to control the repetition

Need to manage i, no need of n

algorithms

$$e^x = \underset{t_0}{1} + \underset{t_1}{x} + \underset{t_2}{x^2/2!} + \underset{t_3}{x^3/3!} + \ldots \qquad \ldots$$

Use x, t, ex, p, i
Input x and p
t ← 1
ex ← t
i ← 1
Repeat if the absolute value of t/ex >= p
    t ← t * x / i
    ex ← ex + t
    i ← i + 1
Output ex, i

e^x with MacLaurin

algo to R prog

```
Use x, t, ex, p, i
Input x and p
t ← 1
ex ← t
i ← 1
Repeat if the absolute value of t/ex >= p
    t ← t * x / i
    ex ← ex + t
    i ← i + 1
Output ex, i
```

```
cat("Computation of exponential with
MacLaurin series\n")
x <- as.double(readline(
  "Insert x "))
p <- as.double(readline(
  "Insert the precision "))
t <- 1
ex <- t
i <- 1
while(abs(t/ex) >= p){
  t <- t * x / i
  ex <- ex + t
  i <- i + 1
}
cat("Exponential of ",x," with ",i,
    " MacLaurin terms: ",ex,"\n")
```

Recap – from algo to R – Repetition 1

algo to R prog

```
repeat varying i from a startV
       value to an endV value

  something

  something

something out of the repetition


i assumes all the consecutive values from
startV to endV
```

```
for (i in startV:endV){
  something
  something
}
something out of the repetition
alternative solution
i <- startV
while (i <= endV){
  something
  something
  i <- i + 1
}
something out of the repetition
```

Recap – from algo to R – Repetition 2

algo to R prog

```
repeat varying i on all the
              positions of vector a
  something
  something
something out of the repetition
```

i assumes all the consecutive values from 1 to length of a

```
for (i in seq_along(a)){
  something
  something
}
something out of the repetition
```

alternative solution

```
i <- 1
while (i <= length(a)){
  something
  something
  i <- i + 1
}
something out of the repetition
```

Vector

Several values on one variable

- A sequence of values of the same type
- A single variable name groups all the values
- The variable can be used *as a single object*
  - copy
  - manipulate
  - algebraic operations
- The single values can be accessed by an *index expression*
  - an integer ranging from 1 to the number of elements in the sequence
- The single values can also be accessed by a *name*, if names are defined

Operations with vectors

- vector creation/assignment

```
v <- c(1,4,12,7,5,2,9)
```

- assigning names

```
names(v) <- c("Monday","Tuesday","Wednesday","Thursday","Friday",
   "Saturday","Sunday")
```

- accessing one element

```
v[3]
v["Wednesday"]
```

- initialize an empty vector

```
v <- c()
```

- concatenate two vectors v1 and v2

```
v <- c(v1,v2)
```

- vector length

```
length(v)
```

Computations with vectors

- all arithmetic operations on vectors are intended element by element
- if lengths are different, the shortest one is replicated as necessary

```
> u <- c(3,6,2,7)
> v <- c(5,3,2,9)
> u+v
[1]  8  9  4 16
> w<-c(u,v)
> v+w #v=(5,3,2,9,5,3,2,9)
      #w=(3,6,2,7,5,3,2,9)
8 9 4 16 10 6 4 18
10*u # (10,10,10,10)*(3,6,2,7)
[1] 30 60 20 70
```

replicated

scalar

Accessing a vector **`v[<index expression>]`** (1)

- A positive integer **`i`**
  - If **`i`** is *not greater than* **`length(v)`**
    - **`v[i]`** is the i-th element of **`v`**
  - If **`i`** is *greater than* **`length(v)`**
    - **`v[i]`** is the **`NA`**
- A negative integer **`-i`**
  - If **`i`** is *not greater than* **`length(v)`**
    - **`v[-i]`** is all the elements of **`v`** excluded the i-th element
  - If **`i`** is *greater than* **`length(v)`**
    - **`v[-i]`** is the entire **`v`**

Accessing a vector **v[<*index expression*>]** (2)

- A vector of positive integers
  - The components of v corresponding to the integers are extracted
- A vector of booleans
  - **`v[c(T,F,F,T)]`** is the vector of the elements of **`v`** corresponding to True
  - If the length of the boolean vector is smaller than **`length(v)`**
    - The index expression elements are replicated to make the lengths equal
  - If the length of the index expression greater than **`length(v)`**
    - A number of **`NA`** values is appended to make the lengths equal

Reading from file - 1

- **Line-by-Line Reading**
- **readLines()**
- Reads file as character vector (one element per line)
- Useful for:
  - Log files
  - Raw text processing
  - Parsing custom formats

Reading from file - 2

- **read.table()**
- Most general-purpose text reader.
- Handles whitespace-delimited data by default.
- Key arguments:
  - file
  - header = TRUE/FALSE
  - sep (e.g., ",", "\t")
  - stringsAsFactors
  - na.strings

many others to come

Conditional execution

Conditional execution

- I need to do something only if some condition is true
- e.g.:
  - **a)** **if** there is no milk in the fridge, **then** buy milk
  - **b)** **if** there is milk in the fridge, **then** pour milk in a glass and drink milk, **else** go to the cornershop and buy milk
  - **c)** **if** you want to drink milk, **then if** there is no milk in the fridge **then** go to the cornershop, buy milk and come back home; pour milk in a glass and drink milk

Algorithm with conditional execution

a) if condition
    action

b) if condition
    action1
  else
    action2

c) if condition1
    if condition2
        action1
    action2

a	*execution*
condition true	action
condition false	-

b	*execution*
condition true	action 1
condition false	action 2

c	*condition 2 true*	*condition 2 false*
condition 1 true	action 1 action 2	action 2
condition 1 false	-	-

Example with conditionals:

what kind of triangle?

- Given three numbers A, B, C in non-decreasing order
  - A ≤ B and B ≤ C
- if the numbers are the measures of the sides of a triangle, what kind of triangle is?

- input A, B and C
- if A+B < C
  - Output "it isn't a triangle"
- else
  - if A equal to C
    - Output "it is equilateral"
  - else
    - if A = B or B = C
      - Output "it is isosceles"
    - else
      - Output "it is scalene"

Robust version

algo to R prog

```
input A, B and C

if A>B swap A and B

if C<A A<-C B<-A C<-B
else if (C<B) swap B and C

if A+B < C
    Output it isn't a triangle

else
    if A equal to C
        Output it is equilateral
    else
        if A = B or B = C
            Output it is isosceles
        else
            Output it is scalene
```

```
A <- as.double(readline("Insert number :"))

B <- as.double(readline("Insert number "))

if (B<A){
   t<-A
   A<-B
   B<-t
}

# now A<=B

C <- as.double(readline("Insert number "))

if (C<A){

   t<-C

   C<-B

   B<-A

   A<-t

} else {

   if (C<B){

      t<-C

      C<-B

      B<-t

}

#now A<=B B<=C

... as before
```

Conditional execution in R

```
if (logical expression){
   …
} else {
  ...
}
```

N.B.: the curly braces are optional if an optional path contains a single statement

What kind of triangle? R solution

```
A <- as.double(readline("Insert number :"))
B <- as.double(readline(paste(
          "Insert number not less than ",A," : ")))
C <- as.double(readline(paste(
          "Insert number not less than ",B," : ")))
if (A + B < C) print("It isn't a triangle") else
  if (A == C) print("It is equilateral") else
    if (A == B || B == C) print("It is isosceles") else
      print("It is scalene")
```

Pay attention to braces (a.k.a. *curly braces* or curly brackets)

algo to R prog

```
a <- 6
b <- 7
if (a < b) {
  print("a is smaller than b")
} else
{
  print("a is not smaller than b")
}
```

The sequence is one line only

```
a <- 6
b <- 7
if (a<b)
  print("a is smaller than b") else
  print("a is not smaller than b")
```

It can also be written without braces, but *else* must be in the same line as the single statement

Pay attention to braces

algo to R prog

```
x <- as.double(readline(
         "Insert number "))
if (x < 0)
  print("Changing sign of x")
  x <- -x
print(x)
```

Regardless the indentation, without braces the block ruled by if is a single statement
This is executed regardless the sign of x

The value shown is always the input with changed sign

```
x <- as.double(readline(
         "Insert number "))
if (x < 0){
  print("Changing sign of x")
  x <- -x
}
print(x)
```

In this case the value shown is always positive

See exercises

- Translate to R the intersection of unsorted lists
- Translate to R the intersection of sorted lists
- Left triangle
- Right triangle
- Tree
- Check input range

Left triangle

```
1
1    2
1    2    3
1    2    3    4
1    2    3    4    5
```

```
# ***Left triangle***
# Use
# - i, j : indexes
# - n: size
# - width: space for each number
# Algorithm:
# repeat varying i from 1 to n
#   repeat varying j from 1 to i
#     output j using width spaces
#         in current line
#   go to new line
```

```
width = 2
n = 5
for (i in 1:n){
  for (j in 1:i){
    cat(format(j, width = width))
  }
  cat("\n")
}
```

Right triangle

```
        1
      1 2
    1 2 3
  1 2 3 4
1 2 3 4 5
```

Tree

```
        1
      1 2 1
    1 2 3 2 1
  1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

Loops again: repeat

- the **repeat** construct introduces a block that is repeatedly executed
- the block must include an if statement to catch the ending condition and to execute, in this case, a break
- the break causes a *jump* out of the repetition

```
repeat {
  …
  if (end condition)
    break
}
```

An example with **repeat**

- repeat is useful when we need to execute the repetition at least once
- read a value checking its range, if the vaule is out of range, read again

```
N1 <- 10 # lower bound allowed
N2 <- 20 # upper bound allowed
repeat {
  n <- as.integer(
         readline(
           paste(
            "Insert a number between",
            N1," and",N2)))
  if (N1 <= n & n <= N2)
    break # if n is in range, loop ends
  cat("value out of range\n")
} # end of repeat
```

Looking for a value in a vector (i)

algorithms

- find what position of vector *v* has the same value as the variable *target*
- scan the components of *v* and stop when a value equal to *target* is found, returning the position where it has been found
- if the value is not found, return the position 0

Looking for a value in a vector (ii)

algo to R prog

```
use v:vector of values,
      target: value to be found
      found: integer, 0 if not found,
                between 1 and length of v
                if found
      i: index on v

input target

set found to 0

repeat varying i from 1 to length of v
    if the position i of v is equal to
    target
        set found to i and stop repeating

if found is zero display not found

else display the value of found
```

```
colors <- c("Red", "Blue", "Green", "Orange")
target <- readline("Insert target color ")
found <- 0
for (i in seq_along(colors)){
  if (colors[i] == target){
    found <- i
    break
  }
}
cat(target)
if (found == 0){
 cat(" not found")
} else {
cat(" found in position ",found)
}
```

Jumps out of the current repetition

from break jumps here

Looking for the values of a vector satisfying a condition

- For example, given v `<- c(3,7,12,9)` look for the elements greater than 6

Tentative Algorithm

Consider all the elements of v and output those that satisfy the condition

Looking for the values of a vector satisfying a condition

- For example, given v `<- c(3,7,12,9)` look for the elements greater than 6

Tentative Algorithm

Consider all the elements of V and output those that satisfy the condition

Not enough specification

Looking for the values of a vector satisfying a condition

algo to R prog

```
Use
v:  vector of values,
condition: a logic expression with v
v_s: the vector of the elements of v
     satisfying the condition
i, is: indexes
Algorithm
create vs as empty vector of the same kind of v
set is to 0
repeat varying i from 1 to length of v
    if the position i of v satisfies the condition
        increment is by 1
        copy the i-th position of v to the
                 is-th position of vs
display vs
```

```
v <- c(3,7,12,9)
# condition v > 6
vs <- vector(mode = mode(v))
is <- 0
for (i in seq_along(v)){
  if (v[i] > 6){
    is <- is + 1
    vs[is] <- v[i]
  }
}
print(vs)
```